Started on	Tuesday, 3 May 2022, 2:36 PM
State	Finished
Completed on	Sunday, 8 May 2022, 5:07 PM
Time taken	5 days 2 hours
Marks	8.00/8.00
Grade	**10.00** out of 10.00 (**100**%)

Question **1**

Correct

Mark 1.00 out of 1.00

Write a C program which calculates and prints the number of whole hours and minutes in a given number of minutes.

There are 60 minutes in an hour. You will need to use the integer division operator **/** to calculate the number of whole hours and the modulus (remainder) operator **%** to calculate the number of remaining minutes. The output of your program should have the following format.

```
250 minutes is equivalent to 4 hours and 10 minutes.
```

Please do not change the provided code.

For example:

Test	Input	Result
number_of_minutes = 250	250	250 minutes is equivalent to 4 hours and 10 minutes.
number_of_minutes = 400	400	400 minutes is equivalent to 6 hours and 40 minutes.

Answer: (penalty regime: 0, 0, 5, 10, 15, 20, 25, 30, 35, 40, 45, 50 %)

Reset answer

```c
#include <stdio.h>

int main(void)
{
    int number_of_minutes;
    scanf("%d", &number_of_minutes);

    double hours = number_of_minutes / 60;
    double minutes = number_of_minutes % 60;

    printf("%d minutes is equivalent to %.0f hours and %.0f minutes.", number_of_minutes, hours, minutes);
    return 0;
}
```

	Test	Input	Expected	Got	
✔	number_of_minutes = 250	250	250 minutes is equivalent to 4 hours and 10 minutes.	250 minutes is equivalent to 4 hours and 10 minutes.	✔
✔	number_of_minutes = 400	400	400 minutes is equivalent to 6 hours and 40 minutes.	400 minutes is equivalent to 6 hours and 40 minutes.	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Question **2**

Correct

Mark 1.00 out of 1.00

Write a C program which does the following:

- Obtains two items of input from the user (there is no prompt provided) - a number and a character. The number will be a valid integer.
- Then it prints the character followed immediately by the number and then the character is printed a second time.

Two example outputs are shown below.

For example:

Input	Result
3 +	+3+
42 *	*42*

Answer: (penalty regime: 0, 0, 5, 10, 15, 20, 25, 30, 35, 40, 45, 50 %)

Reset answer

```c
#include <stdio.h>

int main(void)
{
    float num;
    scanf("%f", &num);
    char letter;
    scanf("%s", &letter);

    printf("%c%.0f%c", letter, num, letter);
    return 0;
}
```

	Input	Expected	Got	
✔	3 +	+3+	+3+	✔
✔	42 *	*42*	*42*	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Question **3**

Correct

Mark 1.00 out of 1.00

Write a C program which does the following:

- Calculate the length of the given strings, namely **str1, str2,** using a function **strlen()** (this function takes a string as an argument and returns its length) and prints the length of these strings. The combined length of the strings will be less than 40 characters long.
- Concatenates **str1** and **str2** using a function **strcat()** (It takes two arguments, i.e, two strings, and stores the resultant concatenated string in the first string specified in the argument.), then print the concatenated string.
- Prints the length of **str1** and **str2** again.

One example output is shown below. The output of your program should have the same format. Given that **str1** is "I_like_" and **str2** is "C_Programs!"

```
The length of str1 is: 7
The length of str2 is: 11
I like C Programs!
The length of str1 is: 18
The length of str2 is: 11
```

Please do not change the provided code.

N.B. The solution will fail if the C compiler generates any warnings. So please check before answering that no warnings are created.

For example:

Test	Input	Result
str1[40]="I_like_" str2[40]="C_Programs!"	I_like_ C_Programs!	The length of str1 is: 7 The length of str2 is: 11 I_like_C_Programs! The length of str1 is: 18 The length of str2 is: 11

Answer: (penalty regime: 0, 0, 5, 10, 15, 20, 25, 30, 35, 40, 45, 50 %)

Reset answer

```c
#include <stdio.h>
#include <string.h>

int main(void)
{

    char str1[40];
    char str2[40];

    scanf("%s", str1);
    scanf("%s", str2);
    int a = strlen(str1);
    int b = strlen(str2);

    strcat(str1, str2);

    printf("The length of str1 is: %i\n", a);
    printf("The length of str2 is: %i\n", b);
    printf("%s\n", str1);

    a = strlen(str1);
    printf("The length of str1 is: %i\n", a);
    printf("The length of str2 is: %i\n", b);
```

	Test	Input	Expected	Got	

✔	`str1[40]="I_like_"`
	`str2[40]="C_Programs!"`
		`I_like_`
		`C_Programs!`
			`The length of str1 is: 7`
			`The length of str2 is: 11`
			`I_like_C_Programs!`
			`The length of str1 is: 18`
			`The length of str2 is: 11`
				`The length of str1 is: 7`
				`The length of str2 is: 11`
				`I_like_C_Programs!`
				`The length of str1 is: 18`
				`The length of str2 is: 11`
					✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Question **4**

Correct

Mark 1.00 out of 1.00

Two C functions **sortInDescend_1()** and **sortInDescend_2()** are given. **sortInDescend_1()** is used for call-by-value and **sortInDescend_2()** is used for call-by-reference.

Please write a C program which does the following:

- Using call-by-value to execute **sortInDescend_1()** and call-by-reference to execute **sortInDescend_2()** , print both resultant outputs.

Given that variables **a1** = 3, **a2** = 1, **a3** = 2, one example output is shown below.

```
After calling sortInDescend_1(), a1 is 3, a2 is 1, a3 is 2
After calling sortInDescend_2(), a1 is 3, a2 is 2, a3 is 1
```

For example:

Test	Input	Result
a1 is 3, a2 is 1, a3 is 2	3 1 2	After calling sortInDescend_1(), a1 is 3, a2 is 1, a3 is 2 After calling sortInDescend_2(), a1 is 3, a2 is 2, a3 is 1

Answer: (penalty regime: 0, 0, 5, 10, 15, 20, 25, 30, 35, 40, 45, 50 %)

Reset answer

```c
#include <stdio.h>
#include <string.h>

void swap(int* i, int* j) {
  int temp;
  temp = *i;
  *i = *j;
  *j = temp;
}

void sortInDescend_1(int var1, int var2, int var3){
    if (var1 < var2){
        swap(&var1, &var2);
    }
    if (var1 < var3){
        swap(&var1, &var3);
    }
    if (var2 < var3){
        swap(&var2, &var3);
    }
}

void sortInDescend_2(int *var1, int *var2, int *var3){
```

	Test	Input	Expected	Got	
✔	a1 is 3, a2 is 1, a3 is 2	3 1 2	After calling sortInDescend_1(), a1 is 3, a2 is 1, a3 is 2 After calling sortInDescend_2(), a1 is 3, a2 is 2, a3 is 1	After calling sortInDescend_1(), a1 is 3, a2 is 1, a3 is 2 After calling sortInDescend_2(), a1 is 3, a2 is 2, a3 is 1	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Question **5**

Correct

Mark 1.00 out of 1.00

Write a C program which does the following:

- Calculate the factorial of n (also known as n!). **n** will be given and can be seen in the template.
- **while** or **do...while** loops are recommended. Please note that the condition in a **while** is slightly different from the one in **do...while**
- Print the result.

Given that n = 3, the output example will be:

```
The result is 6
```

The value of n will be between 1 and 12.
The factorial of n is 1 * 2 * 3 * ... * (n-2) * (n-1) * n

For example:

Test	Input	Result
n=3	3	The result is 6

Answer: (penalty regime: 0, 0, 5, 10, 15, 20, 25, 30, 35, 40, 45, 50 %)

Reset answer

```c
#include <stdio.h>

int main(void)
{
    int n, i;
    int factorial = 1;
    scanf("%d", &n);

    for (i = 1; i <= n; i++)
    {
        factorial *= i;
    }

    printf("The result is %d", factorial);

    return 0;
}
```

	Test	Input	Expected	Got	
✔	n=3	3	The result is 6	The result is 6	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Question **6**

Correct

Mark 1.00 out of 1.00

The Body Mass Index (BMI) a convenient rule of thumb used to broadly categorize a person as *underweight*, *normal weight*, *overweight*, or *obese* based on tissue mass (muscle, fat, and bone) and height. It can be obtained with the following formula:

BMI = weight / (height*height)

with the BMI in kg/m^2, the height in *m* and the mass in *kg*.

Commonly accepted BMI ranges are:

- underweight (< 18),
- normal weight (>=18 and <25),
- overweight (>=25 and <30), and
- obese (>= 30).

Complete the following C program, which is

- taking the height and weight of a person as **command-line arguments**,
- calculating the BMI from the height and weight values,
- checking in what BMI range the person is and printing out the height, weight, BMI and category.

You can consider that the user will always enter **float values** for the height and weight values.

See the examples for the expected format of the output.

Hint: The function `atof()` declared in `stdlib.h` will be useful.

For example:

Test	Result
`./prog 1.85 80`	`height = 1.850000, weight = 80.000000, bmi = 23.374725, category = normal weight`
`./prog 1.75 90`	`height = 1.750000, weight = 90.000000, bmi = 29.387754, category = overweight`

Answer: (penalty regime: 0, 0, 5, 10, 15, 20, 25, 30, 35, 40, 45, 50 %)

Reset answer

```c
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[])
{
    if (argc != 3) {
        printf("Usage: %s height weight\n", argv[0]);

    }
    else {

        float height = atof(argv[1]);
        float weight = atof(argv[2]);
        float bmi = weight / (height * height);

        if (bmi < 18) {
            printf("height = %.6f, weight = %.6f, bmi = %.6f, category = underweight\n", height, weight, bmi);
        }
        else if (bmi < 25) {
            printf("height = %.6f, weight = %.6f, bmi = %.6f, category = normal weight\n", height, weight, bmi);
        }
        else if (bmi < 30) {
            printf("height = %.6f, weight = %.6f, bmi = %.6f, category = overweight\n", height, weight, bmi);
```

	Test	Expected	Got	

	Test	Expected	Got	
✔	`./prog 1.85 80`	`height = 1.850000, weight = 80.000000, bmi = 23.374725, category = normal weight`	`height = 1.850000, weight = 80.000000, bmi = 23.374725, category = normal weight`	✔
✔	`./prog 1.75 90`	`height = 1.750000, weight = 90.000000, bmi = 29.387754, category = overweight`	`height = 1.750000, weight = 90.000000, bmi = 29.387754, category = overweight`	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Question **7**

Correct

Mark 1.00 out of 1.00

Write a C program which reads each line of a file and displays the line.

The file consists of several rows.

- Each row has two fields.
- The fields are separated by a space character.
- One field is a string and the other is an integer

E.g. The content of this file could be like :

```
John 27
Jack 28
```

After reading the file, print them line by line.

```
John 27
Jack 28
```

N.B. Names will be less than 20 characters long.

For example:

Input	Result
fileIO1	John 27 Jack 28

Answer: (penalty regime: 0, 0, 5, 10, 15, 20, 25, 30, 35, 40, 45, 50 %)

Reset answer

```c
#include <stdio.h>
#include <stdbool.h>
#include <string.h>

int main(void){
    char filename[10];
    scanf("%s",filename);

    FILE *fp;
    fp = fopen(filename,"r");
    if(fp == NULL){
        printf("File not found\n");
        return 0;
    }

    char line[100];
    while(fgets(line,100,fp) != NULL){
        printf("%s",line);
    }

    return 0;
}
```

	Input	Expected	Got	
✔	fileIO1	John 27 Jack 28	John 27 Jack 28	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Question **8**

Correct

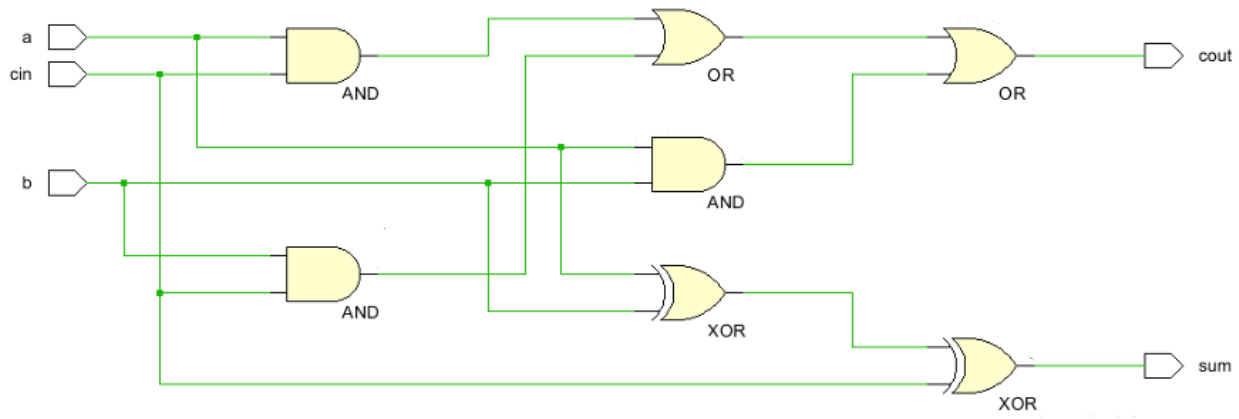
Mark 1.00 out of 1.00

Write a C program to implement a 1-bit full adder using bitwise operations.

The inputs are cin, a and b.

If cin = 1, a = 0, b = 1 the output would be:

```
cout = 1
sum = 0
```



For example:

Test	Input	Result
cin = 1 a = 0 b = 1	1 0 1	cout = 1 sum = 0

Answer: (penalty regime: 0, 0, 5, 10, 15, 20, 25, 30, 35, 40, 45, 50 %)

Reset answer

```
#include <stdio.h>

int main(void){

    int cin,a,b;

    scanf("%d %d %d",&cin,&a,&b);

    int andOne = a & cin;
    int andTwo = b & cin;
    int andThree = a & b;

    int orOne = andOne | andTwo;
    int orTwo = andThree | orOne;

    int xorOne = a ^ b;
    int xorTwo = cin ^ xorOne;

    printf("cout = %d\n",orTwo);
    printf("sum = %d",xorTwo);


    return 0;
```

	Test	Input	Expected	Got	
✔	cin = 1 a = 0 b = 1	1 0 1	cout = 1 sum = 0	cout = 1 sum = 0	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

◄ C Programming Practice

Jump to...

Tutorial 7 ►